# HEART DISEASE ANALYSIS

Team Number: 326
Team Lead: Rajat Kumar
Team Members: Chirag Yadav, Jiyotirmaan Singh

# 1. Heart Disease Analytics Dashboard Using Flask and Tableau

Table of Contents

1. Project Title

2. Project Description

3. Application Scenarios

4. Technical Architecture

5. Prerequisites

6. Prior Knowledge

7. Project Objectives

8. System Workflow

9. Requirement Analysis

10. Environment Setup

11. Core Development

12. Integration

13. Testing

14. Deployment

15. Project Structure

16. Results

17. Advantages & Limitations

18. Future Enhancements

19. Conclusion

20. Appendix

# 2. Project Description

## 2.1 System Overview

The Heart Disease Analytics Dashboard is a web-based analytical platform designed to explore patterns in healthcare datasets related to cardiovascular disease. The system integrates interactive data visualization tools with a web application interface to allow users to analyze medical indicators and identify trends associated with heart disease risk.

The application uses **Flask as the backend framework**, **HTML and CSS for the frontend interface**, and **Tableau for interactive visual analytics**. The dashboard allows users to view different health indicators such as age, BMI, physical activity, smoking habits, and demographic distributions.

## 2.2 Problem Domain

Heart disease is one of the leading causes of death globally. Healthcare datasets contain large amounts of information regarding patient health metrics. However, analyzing this data manually can be difficult and inefficient.

This project aims to simplify healthcare data analysis by providing an **interactive visualization platform** where users can explore patterns and correlations in the dataset.

## 2.3 Core Technology Used

The project is built using the following technologies:

- Python
- Flask Web Framework
- HTML and CSS
- Tableau Data Visualization
- Healthcare Dataset

## 2.4 Key Capabilities

The system provides the following capabilities:

- Interactive dashboard visualization
- Analysis of health indicators
- Summary statistics for quick insights
- User-friendly web interface

## 2.5 Target Users

The system can be useful for:

- Healthcare researchers
- Data analysts
- Medical students
- Academic researchers
- Educational institutions

## 2.6 Real-World Relevance

Data visualization tools can help healthcare professionals understand risk factors associated with cardiovascular diseases. By analyzing health indicators visually, patterns and correlations can be identified more effectively.

# 3. Application Scenario

The Heart Disease Analytics Dashboard can be applied in several real-world scenarios where healthcare data needs to be analyzed and interpreted effectively. By presenting complex datasets through interactive visualizations, the system helps users understand patterns related to cardiovascular health.

## 3.1 Healthcare Research and Data Analysis

One of the primary applications of this dashboard is in healthcare research. Medical researchers and data analysts often work with large datasets containing various health indicators such as age, BMI, lifestyle habits, and medical conditions. Analyzing such datasets manually can be difficult and time-consuming.

The dashboard simplifies this process by presenting the data through interactive charts and graphs. Researchers can quickly identify relationships between variables, such as how age, smoking habits, or physical activity may influence the occurrence of heart disease. This allows researchers to gain insights more efficiently and supports further medical analysis.

## 3.2 Educational and Academic Use

The system can also be used in academic environments for teaching purposes. Students studying data analytics, computer science, or healthcare informatics can use the dashboard to understand how real-world datasets are analyzed using visualization tools.

By interacting with the charts and exploring the dataset, students can observe how different health indicators relate to heart disease risk. The project also demonstrates how web technologies like Flask can be integrated with data visualization platforms such as Tableau to build interactive analytical systems.

## 3.3 Public Health Awareness

Another important use case of this project is increasing awareness about cardiovascular health. Visual representations of healthcare data can help individuals better understand how lifestyle factors and demographic variables contribute to heart disease.

Public health organizations can use similar dashboards to present health statistics to the public in a clear and understandable way. This can support awareness campaigns and encourage people to adopt healthier habits that reduce the risk of heart disease.

# 4. Technical Architecture

## 4.1 High-Level Architecture

The system follows a **three-layer architecture**:

1. Frontend Layer
2. Backend Layer
3. Visualization Layer

## 4.2 Frontend Layer

The frontend is built using:

- HTML
- CSS

This layer is responsible for displaying the user interface, including the homepage and dashboard layout.

## 4.3 Backend Layer

The backend is implemented using **Flask**, a lightweight Python web framework. Flask handles:

- Routing
- Page rendering
- Integration with visualization components

## 4.4 Visualization Layer

The visualization layer uses **Tableau dashboards** embedded within the web application through an iframe. Tableau processes the dataset and generates interactive charts.

## 4.5 Deployment Environment

The system runs locally on a development server using Flask and can be accessed through a web browser.

# 5. Prerequisites

## 5.1 Software Requirements

The following software is required to run the project:

- Python 3.x
- Flask
- Web browser (Chrome, Firefox, etc.)
- Visual Studio Code or any code editor

## 5.2 Libraries / Frameworks / Dependencies

Required Python package:

- Flask

## 5.3 Hardware Requirements

Minimum system requirements:

- 4 GB RAM
- Modern web browser
- Standard personal computer or laptop

# 6. Prior Knowledge

To understand and implement this project effectively, certain basic technical knowledge is helpful. These concepts provide the foundation required for developing the web application and integrating the data visualization components.

**Basic Python Programming**
A basic understanding of Python programming is required to develop and run the Flask-based web application. Python is used to handle routing, server execution, and overall application logic.

**Web Development Fundamentals (HTML and CSS)**
Knowledge of HTML and CSS is necessary to design the structure and appearance of the web pages. These technologies are used to create the user interface for the homepage and dashboard layout.

**Understanding of Flask Framework**
Familiarity with the Flask framework helps in understanding how web routes, templates, and server-side logic work together to serve the web application.

**Basic Knowledge of Data Visualization**
Understanding the fundamentals of data visualization helps users interpret the charts and graphs presented in the dashboard and understand how data insights are derived.

**Understanding of Healthcare Datasets**
Basic knowledge of healthcare-related datasets and common health indicators is useful for interpreting the information presented in the dashboard and understanding patterns related to heart disease.

# 7. Project Objective

## 7.1 Technical Objectives

- Build a web application using Flask
- Integrate a Tableau dashboard within the application
- Provide interactive data visualization

## 7.2 Performance Objectives

- Ensure the dashboard loads efficiently
- Maintain smooth user interaction

## 7.3 Deployment Objectives

- Run the application locally using Flask
- Provide easy access through a web browser

## 7.4 Learning Outcomes

Through this project, the team gained experience in:

- Web application development
- Data visualization techniques
- Dashboard design
- Integration of external visualization tools

# 8. System Workflow

The system follows a structured workflow that allows users to access and analyze healthcare data through the web application. The process begins when the user opens the application in a web browser.

First, the user accesses the web application through the Flask server. The homepage is displayed, providing an introduction to the Heart Disease Analytics Dashboard and explaining the purpose of the system.

Next, the user clicks the **"Open Dashboard"** button available on the homepage. This action sends a request to the Flask backend, which processes the request and routes the user to the dashboard page.

Once the dashboard page loads, the embedded Tableau visualization dashboard is displayed within the webpage. The dashboard presents various charts and graphs that represent healthcare data related to heart disease.

Finally, users can interact with these visualizations to explore relationships between different health indicators such as age, gender, and lifestyle factors. This interaction allows users to analyze patterns and gain insights from the dataset in a clear and intuitive way.

4.

# 9. Requirement Analysis

## 9.1 Problem Definition

Healthcare datasets contain multiple variables affecting cardiovascular health. The challenge is to analyze this data efficiently.

## 9.2 Functional Requirements

- Display homepage
- Display analytics dashboard
- Show health data visualizations

## 9.3 Non-Functional Requirements

- Simple user interface
- Fast loading dashboard
- Easy navigation

## 9.4 System Design Decisions

Flask was chosen for backend development due to its lightweight architecture. Tableau was used for visualization because of its powerful analytics capabilities.

# 10. Environment Setup

## 10.1 Development Environment Setup

The project environment was set up using Python and Flask.

## 10.2 Dependency Installation

```
pip install flask
```

## 10.3 Project Structure Creation

The following folder structure was created:

```
heart_disease_project
│
├── app.py
├── templates
│   ├── index.html
│   └── dashboard.html
└── static
```

## 10.4 Configuration Setup

Flask routes were configured to load the homepage and dashboard pages.

# 11. Core Development

## 11.1 Feature 1 – Homepage

The homepage serves as the entry point of the web application. It provides a brief introduction to the Heart Disease Analytics Dashboard and explains the purpose of the project. The page also includes navigation elements that allow users to access the main analytics dashboard. This interface is designed to be simple and user-friendly, enabling users to quickly understand the purpose of the system and proceed to explore the data visualizations.

## 11.2 Feature 2 – Dashboard Integration

The interactive data visualization dashboard was created using Tableau and integrated into the web application. The Tableau dashboard is embedded within the dashboard page using an iframe, allowing the visualizations to be displayed directly inside the website. This integration enables users to interact with charts and graphs that represent healthcare data and explore different patterns related to heart disease.

## 11.3 Feature 3 – KPI Cards

Key Performance Indicator (KPI) cards were added at the top of the dashboard to provide a quick summary of important statistics from the dataset. These cards help users understand the overall distribution of the data before exploring detailed visualizations.

The KPI cards display the following metrics:

- Total number of patients in the dataset
- Number of heart disease cases
- Number of healthy patients
- Overall risk ratio within the dataset

These summary indicators provide a quick overview of the dataset and enhance the usability of the dashboard.

# 12. Integration

## 12.1 Component Integration

The system integrates multiple components to create a functional analytics platform. The Flask backend is responsible for handling routing and serving the web pages, while the frontend interface displays the user interface using HTML and CSS. The Tableau dashboard is embedded within the dashboard page using an iframe, allowing interactive visualizations to be displayed directly within the web application. This integration ensures that users can access data visualizations seamlessly through the web interface.

## 12.2 Performance Optimization

Basic performance optimization techniques were applied to ensure smooth loading of the dashboard. The layout structure of the webpage was optimized to reduce unnecessary elements and improve page rendering. This helps the dashboard load more efficiently and provides a better user experience when accessing the visualizations.

## 12.3 Error Handling & Validation

Basic validation mechanisms were implemented to ensure that the application functions correctly during user navigation. Flask routing ensures that the correct pages are loaded when users access different sections of the application. This helps prevent navigation errors and ensures that the dashboard and homepage load properly.

# 13. Testing

## 13.1 Test Cases

Several test cases were performed to ensure that the web application and dashboard function correctly. These tests verified whether the application loads properly, routes function as expected, and the embedded Tableau dashboard displays the visualizations without errors.

| Test Case ID | Input | Expected Output | Status |
|---|---|---|---|
| TC01 | Open homepage | Homepage loads successfully | Passed |
| TC02 | Click dashboard button | Dashboard page opens | Passed |
| TC03 | Load Tableau dashboard | Visualizations appear correctly | Passed |

## 13.2 Unit Testing

Unit testing was performed on the individual components of the system. The Flask routes were tested to ensure that the correct pages are rendered when users navigate through the application. Basic functionality such as loading the homepage and dashboard pages was verified.

## 13.3 Integration Testing

Integration testing was conducted to ensure that different components of the system work together properly. The Flask web application and the embedded Tableau dashboard were tested together to confirm that the visualization loads correctly within the web interface and functions without compatibility issues.

## 13.4 User Acceptance Testing

User acceptance testing was carried out to evaluate whether the system meets the intended project requirements. Users interacted with the dashboard and confirmed that the interface is easy to navigate and that the visualizations clearly represent healthcare data and trends related to heart disease.

# 14. Deployment

## 14.1 Deployment Architecture

The application follows a simple deployment architecture based on a Flask web server. The Flask framework handles routing and serves the web pages of the application. The frontend interface, developed using HTML and CSS, is displayed in the web browser, while the Tableau dashboard is embedded within the application using an iframe.

When the user accesses the application through a browser, the request is processed by the Flask server, which then loads the corresponding webpage and displays the interactive visualizations.

## 14.2 Hosting Platform

Currently, the application is deployed locally on a Flask development server. This means the system runs on the developer's machine and can be accessed through a web browser using the local server address.

In future implementations, the system can be deployed on cloud platforms such as AWS, Google Cloud, or Heroku to make it accessible over the internet.

## 14.3 Deployment Steps

The following steps are required to run the application:

1. Install the required dependencies, including the Flask framework.
2. Navigate to the project directory containing the Flask application.
3. Run the Flask application using the main Python file.
4. Open a web browser and access the application using the local server address.

Example command:

```
python app.py
```

After running the application, the dashboard can be accessed through the browser at:

```
http://127.0.0.1:5000
```

This launches the web interface and allows users to interact with the heart disease analytics dashboard.

# 15. Project Structure

The project follows a simple and organized folder structure that separates the web application components, dataset, visualizations, and documentation. This structure helps maintain clarity and makes the project easier to understand and manage.

```
Heart_Disease_Analysis
│
├── Website_Screenshots
│   ├── Screenshot1.png
│   ├── Screenshot2.png
│   └── Screenshot3.png
│
├── data
│   └── Heart_new21.xlsx
│
├── heart_disease_project
│   │
│   ├── static
│   │   ├── blr.jpeg
│   │   └── style.css
│   │
│   ├── templates
│   │   ├── index.html
│   │   └── dashboard.html
│   │
│   └── app.py
│
├── tableau
│   └── Book11.twbx
│
└── README.md
```

# 16. Results

The developed system successfully provides an interactive dashboard for analyzing healthcare data related to heart disease. By integrating the Tableau visualization dashboard within a Flask-based web application, the platform allows users to explore and interpret healthcare data through clear graphical representations.

The dashboard presents several important analytical visualizations that highlight relationships between different health indicators and heart disease occurrence. Key outputs of the system include:

- **Gender vs Heart Disease Visualization:**
  This chart compares the distribution of heart disease cases between male and female patients, helping users understand how gender may influence the occurrence of cardiovascular conditions.

- **Age vs Heart Disease Analysis:**
  This visualization examines the relationship between different age groups and the prevalence of heart disease, allowing users to observe how risk may increase with age.

- **Impact of Smoking and Alcohol Consumption:**
  The dashboard analyzes how lifestyle factors such as smoking and alcohol consumption may affect heart disease occurrence, providing insights into behavioral risk factors.

- **Race-wise Distribution of Heart Disease:**
  This chart displays the distribution of heart disease cases across different demographic groups, enabling users to identify variations within the dataset.

Overall, these visualizations help simplify complex healthcare data and allow users to identify patterns and trends that may contribute to better understanding of cardiovascular health.

# 17. Advantages and Limitations

## 17.1 Advantages

- ### Simple and Interactive Interface
  The dashboard provides a clean and user-friendly interface that allows users to easily navigate through the system and explore the visualizations. The interactive charts help users understand complex healthcare data more effectively without requiring advanced technical knowledge.

- ### Easy Integration of Visualization Tools
  The system demonstrates how visualization tools like Tableau can be easily integrated with a web application built using Flask. This integration allows powerful data analysis capabilities to be combined with a simple web-based interface.

- ### Clear Representation of Healthcare Data
  The use of charts, graphs, and visual dashboards helps present healthcare data in a clear and understandable format. This makes it easier for users to identify patterns, trends, and relationships between different health indicators.

## 17.2 Limitations

- ### Limited Dataset Size
  The current implementation uses a relatively small dataset for analysis. A larger and more diverse dataset would provide more comprehensive insights and improve the overall reliability of the analysis.

- ### No Predictive Analytics Implemented
  The system currently focuses only on data visualization and does not include predictive models. Integrating machine learning algorithms in the future could allow the system to predict heart disease risk based on patient data.

- ### Currently Runs Only on Local Server
  At present, the application runs on a local Flask development server. This limits accessibility since users must run the application on their own machine. Deploying the system to a cloud platform would make it accessible to users over the internet.

# 18. Future Enhancements

Future improvements can further expand the capabilities and usefulness of the Heart Disease Analytics Dashboard. Some potential enhancements include the following:

## Integration of Machine Learning Models for Heart Disease Prediction

Machine learning algorithms can be integrated into the system to predict the risk of heart disease based on patient health indicators such as age, BMI, physical activity, and lifestyle habits. By training predictive models on healthcare datasets, the system could provide automated risk assessments and support early detection of cardiovascular issues.

## Real-time Patient Data Input

Currently, the system analyzes a static dataset. In future versions, the application could allow users or healthcare professionals to enter patient data through a web form. The system would then process this input and generate updated visualizations or risk predictions, making the dashboard more interactive and practical for real-time analysis.

## Cloud Deployment

The application can be deployed on cloud platforms such as AWS, Google Cloud, or Heroku to make it accessible from anywhere through the internet. Cloud deployment would improve scalability, reliability, and availability, allowing multiple users to access the system simultaneously.

## Mobile-Friendly Interface

A responsive design or dedicated mobile interface could be developed so that the dashboard can be easily accessed on smartphones and tablets. This would allow healthcare professionals or users to view analytics and insights conveniently on portable devices.

# 19. Conclusion

The Heart Disease Analytics Dashboard demonstrates how modern web technologies and data visualization tools can be effectively combined to analyze healthcare datasets. By integrating Flask for the web interface and Tableau for interactive visualizations, the system provides a structured platform for exploring patterns related to cardiovascular health.

The project enables users to examine important health indicators such as age, lifestyle factors, and demographic information through clear and interactive visual representations. These visualizations help simplify complex datasets and allow users to identify potential trends and risk factors associated with heart disease.

Overall, the project highlights the importance of data-driven analysis in healthcare and illustrates how visualization platforms can support better understanding of medical data. With further enhancements such as predictive analytics and cloud deployment, the system could be expanded into a more advanced healthcare analytics tool in the future.

# 20. Appendix

## GitHub Repository:

https://github.com/jiyotirmaansingh/Heart_Disease_Analysis/tree/main

## Live Demo:

https://drive.google.com/file/d/1pP26A2WB3Bnovb1_h9KMADf88XlhfGO0/view?usp=sharing

## DataSet:

Healthcare dataset used for heart disease analysis.

Thank You!